

Relatório Redes

ALUNO: Thainan André da Silva

SUMÁRIO

O trabalho em questão trata-se da implementação de um cliente e servidor que sigam o protocolo UDP, com o objetivo principal de por em prática a implementação do protocolo de Transferência Confiável de Dados, o RDT 2.0 visto em KUROSE (2013).

ESTRUTURA DE DADOS

A estrutura de dados utilizada para armazenar os postos de combustíveis, foi implementada uma KD-Tree, com nome auto-descritivo é uma árvore K-Dimensional. A KD-Tree armazena seus dados quase que semelhantemente a uma Binary Tree, porém com uma diferença que a torna única: a cada nível que descemos na árvore, ela muda o campo utilizado para verificação.

SERVIDOR

No servidor, foram utilizadas threads para fazer o atendimento dos clientes, junto de mais três rotinas que auxiliam o seu funcionamento.

- **periodicSavior:** Rotina que faz o salvamento dos dados a cada 5 segundos. Uma única thread fica responsável por ficar armazenando os dados no arquivo para que não sejam armazenados a cada alteração. Possui um semáforo para não causar conflito enquanto faz a escrita dos dados no arquivo.
- **writer:** Rotina chamada quando o cliente entra com uma mensagem de dados (tipo 'D'). Possui semáforos para evitar leitura e escrita de dados incorretos e, ao conseguir entrar na zona crítica, faz a inserção dos dados na KD-Tree e retorna ao cliente que os dados foram inseridos.
- **reader:** Rotina responsável para uma mensagem de pesquisa (tipo 'P'). A reader também possui semáforos, porém bloqueia somente escritores. Ao conseguir entrar em sua zona crítica, faz a busca pelo menor preço do combustível dentro do raio e o retorna para o cliente.

Vale ressaltar que o servidor recebe a mensagem, ACK ou NAK via string.

CLIENTE

No cliente, é lida e validada a linha de dados que será enviada ao servidor e repassada à sua rotina auxiliar chamada de rdtSend.

A rdtSend possui a função de enviar o dado e aguardar o ACK. Caso receba um NAK, é feita a retransmissão do dado repentinamente até receber o ACK confirmando que a mensagem foi recebida corretamente.

DECISÃO DE IMPLEMENTAÇÃO

Na propagação da mensagem com erro, foi decidido implementar de outra maneira, utilizando a checksum do RDT para fazer a validação da mensagem, visando aprender os conceitos do tratamento de

dados corrompidos do RDT 2.0, assim, não utilizando um campo erro dentro do pacote para indicar um erro de bits. A rotina responsável pelo checksum faz um XOR em todos os bytes e retorna o resultado para quem a chamou.

RETRANSMISSÃO DE MENSAGENS

A retransmissão de mensagens ocorre por parte do cliente, assim como no RDT, onde o mesmo envia a mensagem e fica esperando uma resposta de confirmação. Ao receber um NAK, o cliente faz novamente o empacotamento dos dados e reenvia ao servidor. Ao receber um ACK, o cliente prossegue e espera os dados que o servidor irá mandar.

TESTES

1º Teste:

Resultado Cliente:

```
> D 0 5677 -22.001 -54.85746
[RDT_SEND] Enviando pacote...
[ERRO] NAK ou resposta corrompida. Retransmitindo...
[RDT_SEND] Enviando pacote...
[SUCESSO] Operacao confirmada pelo servidor.
|--> Resposta: [SERVIDOR] Dados inseridos.
```

O cliente envia os dados e espera por um ACK. Recebe primeiramente um NAK, fazendo com que reenvie os dados ao servidor. Logo após recebe um ACK e aguarda os dados subsequentes, imprimindo-os posteriormente.

Resultado Servidor:

```
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SERVIDOR] Dados recebidos: 127:D 0 5677 -22.001 -54.8574X
[RDT] Pacote corrompido!
Checksum Recebido: 127, Calculado: 17. Envia NAK.
[SERVIDOR] Dados recebidos: 127:D 0 5677 -22.001 -54.85746
[RDT] Pacote recebido e validado. Enviando ACK e processando...
[WRITER] Dados inseridos.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
```

O servidor recebe os dados, faz o checksum e, verificando que não é o esperado, envia NAK. Logo após, recebe novamente o pacote, faz o checksum e valida, prosseguindo para o processamento dos dados.

2º Teste:

Resultado Cliente:

```
> D 1 3456 -22 -54
[RDT_SEND] Enviando pacote...
[ERRO] NAK ou resposta corrompida. Retransmitindo...
[RDT_SEND] Enviando pacote...
[SUCESSO] Operacao confirmada pelo servidor.
|--> Resposta: [SERVIDOR] Dados inseridos.
```

Mensagem de dados para inserção do valor 3456 no campo Etanol do posto que está localizado na latitude -22 e longitude -54. Recebe primeiramente um NAK, reenviando a mensagem para o servidor. Após o primeiro NAK, recebe um ACK, indicando que a mensagem foi entregue e imprime a resposta.

Resultado Servidor:

```
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SERVIDOR] Dados recebidos: 112:D 1 3456 -22 -5X
[RDT] Pacote corrompido!
Checksum Recebido: 112, Calculado: 28. Envia NAK.
[SERVIDOR] Dados recebidos: 112:D 1 3456 -22 -54
[RDT] Pacote recebido e validado. Enviando ACK e processando...
[WRITER] Dados inseridos.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
```

O servidor recebe os dados, faz o checksum e vê que o pacote está corrompido, enviando de volta um NAK. Recebe os dados novamente, agora intactos, processa-os e faz a inserção dos dados

3º Teste:

Resultado Cliente:

```
> D 0 4567 -22 -54
[RDT_SEND] Enviando pacote...
[ERRO] NAK ou resposta corrompida. Retransmitindo...
[RDT_SEND] Enviando pacote...
[SUCESSO] Operacao confirmada pelo servidor.
|--> Resposta: [SERVIDOR] Dados inseridos.
```

Mensagem de dados para a inserção do valor 4567 no Diesel do posto de combustível que está localizado na latitude -22 e longitude -54. Recebe primeiramente um NAK e reenvia o pacote. Logo após recebe um ACK, imprimindo na tela o conteúdo da resposta.

Resultado Servidor:

```
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SERVIDOR] Dados recebidos: 117:D 0 4567 -22 -5X
[SDT] Pacote corrompido!
Checksum Recebido: 117, Calculado: 25. Envia NAK.
[SERVIDOR] Dados recebidos: 117:D 0 4567 -22 -54
[SDT] Pacote recebido e validado. Enviando ACK e processando...
[WRITER] Dados inseridos.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
```

O servidor verifica que o checksum está incorreto, retornando um NAK ao cliente. Logo ao receber o novo pacote, verifica o checksum novamente e vê que está correto, assim dando continuidade ao processamento do pacote e retornando a resposta ao cliente.

4º Teste:

Resultado Cliente:

```
> P 0 1000 -22.0008 -54.00847
[RD_SEND] Enviando pacote...
[ERRO] NAK ou resposta corrompida. Retransmitindo...
[RD_SEND] Enviando pacote...
[ERRO] NAK ou resposta corrompida. Retransmitindo...
[RD_SEND] Enviando pacote...
[ERRO] NAK ou resposta corrompida. Retransmitindo...
[RD_SEND] Enviando pacote...
[ERRO] NAK ou resposta corrompida. Retransmitindo...
[RD_SEND] Enviando pacote...
[SUCESSO] Operacao confirmada pelo servidor.
|--> Resposta: [SERVIDOR] Melhor preco encontrado: 4.567
```

Mensagem de pesquisa buscando o menor valor no campo Diesel, em um raio de 1000 quilômetros partindo do centro de latitude -22.0008 e longitude -54.00847. Recebe então 3 NAKs consecutivos, reenviando o pacote 3 vezes. Na 4ª tentativa recebe o ACK e imprime na tela o resultado.

Resultado Servidor:

```
[SERVIDOR] Arquivo de dados salvo com sucesso.  
[SERVIDOR] Dados recebidos: 83:P 0 1000 -22.0008 -54.0084X  
[RDT] Pacote corrompido!  
Checksum Recebido: 83, Calculado: 60. Envia NAK.  
[SERVIDOR] Dados recebidos: 83:P 0 1000 -22.0008 -54.0084X  
[RDT] Pacote corrompido!  
Checksum Recebido: 83, Calculado: 60. Envia NAK.  
[SERVIDOR] Dados recebidos: 83:P 0 1000 -22.0008 -54.0084X  
[RDT] Pacote corrompido!  
Checksum Recebido: 83, Calculado: 60. Envia NAK.  
[SERVIDOR] Dados recebidos: 83:P 0 1000 -22.0008 -54.00847  
[RDT] Pacote recebido e validado. Enviando ACK e processando...  
[SERVIDOR] Iniciando salvamento periodico
```

O servidor recebe o pacote e por 3 vezes verifica que o dado chegou corrompido, enviando NAK em todas as vezes. Ao validar o dado e ver que estava intacto, dá continuidade e processa-o para gerar o resultado e retorna-lo ao cliente.

5º Teste:

Resultado Cliente:

```
> P 1 1000 -22.02345 -54.857438  
[RDT_SEND] Enviando pacote...  
[ERRO] NAK ou resposta corrompida. Retransmitindo...  
[RDT_SEND] Enviando pacote...  
[SUCESSO] Operacao confirmada pelo servidor.  
|--> Resposta: [SERVIDOR] Melhor preco encontrado: 3.456
```

Mensagem de pesquisa buscando o menor preço no campo Etanol em um raio de 1000 quilômetros partindo de um centro de latitude -22.02345 e longitude -54.857438. Recebe um NAK primeiramente, reenviando o pacote e posteriormente ao receber um ACK imprime o retorno do servidor.

Resultado Servidor:

```
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SERVIDOR] Dados recebidos: 84:P 1 1000 -22.02345 -54.85743X
[SDT] Pacote corrompido!
Checksum Recebido: 84, Calculado: 52. Envia NAK.
[SERVIDOR] Dados recebidos: 84:P 1 1000 -22.02345 -54.857438
[SDT] Pacote recebido e validado. Enviando ACK e processando...
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
```

O servidor recebe e faz a validação dos dados. Ao serem validados, é encaminhado um ACK indicando que os dados chegaram ok e logo após é enviado o resultado da pesquisa feita pelo cliente.

6º Teste:

Resultado Cliente:

```
> P 2 1000 -22.9086908 -54.845785477
[SDT_SEND] Enviando pacote...
[SUCESSO] Operacao confirmada pelo servidor.
|--> Resposta: [SERVIDOR] Nenhum posto encontrado dentro do raio
```

Mensagem de pesquisa para realizar a busca do menor preço do campo Gasolina em um raio de 1000 quilômetros. Recebe de primeira um ACK, porém não encontra nenhum posto com preço para Gasolina dentro do raio pois não foi inserido nenhum valor para a mesma.

Resultado Servidor:

```
[SAVIOR] Arvore de dados salva com sucesso.
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
[SERVIDOR] Dados recebidos: 99:P 2 1000 -22.9086908 -54.845785477
[SDT] Pacote recebido e validado. Enviando ACK e processando...
[SAVIOR] Iniciando salvamento periodico...
[SAVIOR] Arvore de dados salva com sucesso.
```

Servidor recebe os dados e processa-os, validando de primeira e prosseguindo para o processamento destes dados, posteriormente quando concluído retornando ao cliente.

CONCLUSÃO

Este trabalho demonstrou a implementação do protocolo UDP com Transferência Confiável de Dados 2.0 (RDT 2.0) conforme descrito em Kurose (2013). A abordagem da detecção de erros utilizando o checksum, em vez de um campo de erro explícito, também se mostrou eficaz para identificar pacotes corrompidos, permitindo reenvio por parte do cliente até o recebimento de um ACK.

A arquitetura do servidor, com suas threads, rotinas auxiliares (periodicSavior, writer e reader),

implementação de semáforos garantiu a eficiência e a segurança dos dados armazenados na KD-Tree, mesmo sob concorrência.

Portanto, a aplicação prática do RDT 2.0 resulta em um servidor UDP confiável, capaz de lidar com dados corrompidos e retransmissão de mensagens.

REFERÊNCIAS

KUROSE, James - Redes de Computadores e a Internet uma abordagem top-down-Pearson (2013).

<https://archive.org/search.php?query=subject%3A%22James++Redes+de+Computadores+e+a+Internet++uma+abordagem+top-down-Pearson+%282013%29%22>

URL: <https://www.youtube.com/watch?v=k9NON5-OSnc> - (Vídeo com explicação do funcionamento da árvore).

URL: <https://www.inf.ufsc.br/~aldo.vw/estruturas/k-d/arvore-kd.html> - (Link para entendimento dos conceitos da árvore).

URL: https://en.wikipedia.org/wiki/K-d_tree - (Link para entendimento dos conceitos da árvore).

URL: <https://www.geeksforgeeks.org/search-and-insertion-in-k-dimensional-tree/> - (Link para entendimento dos conceitos da árvore).

URL: <https://www.geeksforgeeks.org/deletion-in-k-dimensional-tree/> - (Remoção de um nó na árvore).

URL: https://pt.wikipedia.org/wiki/F%C3%B3rmula_de_haversine - Cálculo da distância entre dois pontos em uma esfera pela fórmula de haversine dadas latitudes e longitudes.

URL: <https://stackoverflow.com/questions/31151032/writing-an-8-bit-checksum-in-c> - Ideia de checksum

URL: <https://cse.usf.edu/~kchrste/tools/checksum.c> - Ideia de checksum

URL: <https://www.b4x.com/android/forum/threads/trying-to-figure-out-bitwise-checksum-code-from-example-in-c.160148/> - Ideia de checksum